A Decentralized LoRa Mesh with On-Device Blockchain and Federated Learning for First-Responder Wearable Monitoring

Kunal Rathod, Abhinav Potharaju, Akshad Jagdale, Ibrahim Bagwan
Department of Artificial Intelligence and Machine Learning
RV College of Engineering, Bengaluru – 560059, India

Prof. Saraswathi Govind Datar
Department of CSE
RV College of Engineering, Bengaluru – 560059, India

***Abstract*—First-responder teams operating in post-disaster environments face a recurring problem: the communications infrastructure they rely on collapses at exactly the moment they need it most. This paper describes a wearable node architecture that sidesteps this dependency by making the mesh network, audit ledger, and anomaly-detection model resident on the devices the responders carry. Each node pairs a Semtech SX1276 LoRa transceiver with an ESP32-C3 microcontroller (320 KB SRAM, 160 MHz RISC-V) to run three co-designed software layers without any external infrastructure. The first layer is a 433 MHz peer-to-peer LoRa mesh configured at spreading factor 10, 125 kHz bandwidth, and coding rate 4/8, which yields a receiver sensitivity of -132 dBm and covers 200–500 m indoors. The second layer is a lightweight Proof-of-Authority blockchain in which each node appends 124-byte packed blocks, linked by a SHA-256 hash chain computed through the mbedTLS library in under 1.2 ms per block. The third layer is a Federated Averaging protocol that exchanges eight-weight logistic regression updates across the mesh rather than raw physiological data; Gaussian differential-privacy noise drawn from the ESP32's hardware entropy source bounds the per-user leakage. An MPU6050 IMU provides freefall-and-impact fall detection; a MAX30102 PPG sensor measures heart rate and SpO2. Validation spans a hybrid testbed of two physical nodes and a 20-node Python simulation connected through an MQTT broker. *[Results to be inserted after hardware experiments.]* The total bill of materials per node is under USD 12.**

***Index Terms*—LoRa, mesh networking, blockchain, federated learning, differential privacy, wearable sensors, first responders, ESP32-C3, SHA-256, anomaly detection**

I. Introduction

The 9/11 World Trade Center response exposed a specific failure mode: police officers received an aerial warning that the North Tower was structurally failing; the 343 firefighters who died did not, because their radios were on incompatible frequencies [1]. Hurricane Katrina in 2005 shut down the New Orleans 911 system for three days. Neither failure was novel. Every major disaster of the past three decades has reproduced the same pattern—high demand for communications coinciding with infrastructure destruction.

Existing wearable monitoring products for emergency personnel address some of this gap but not all of it. Personal alert safety system (PASS) devices detect motion cessation and sound an alarm, but they carry no physiological data and have no network interface. Systems built on FirstNet (4G LTE) or commercial LoRaWAN require functioning base stations within range. When the base station burns or floods, the network ends.

What is missing is a wearable that carries its own network. This paper proposes one. Our system nodes communicate directly over unlicensed 433 MHz LoRa radio in a peer-to-peer mesh, log sensor events in an on-device blockchain whose integrity can be verified by any other node without a server, and train a shared anomaly detector using federated learning so that no raw biometric data leaves the device that measured it. The combined bill of materials is under USD 12 per node, using only commodity parts that do not require export licenses or specialized sourcing channels.

The technical challenge is that LoRa is slow. At spreading factor 10 (SF10), the 124-byte block we transmit takes 1.8 s of channel time. Getting blockchain, federated learning, and sensor sampling to coexist around that airtime constraint on a single core running FreeRTOS required some non-obvious design choices that we document here.

The rest of this paper is structured as follows. Section II surveys prior work. Section III describes the system architecture. Section IV covers the firmware and simulation implementation. Section V presents experimental results. Section VII concludes.

II. Related Work

A. LoRa for Public Safety

Petäjäjärvi et al. measured LoRa at 868 MHz across Finnish countryside and reported reliable coverage to 15 km at SF12 [2]. Bor et al. characterized LoRa network capacity under the Poisson traffic model and showed that capture effect allows

the stronger of two simultaneous transmissions to survive if it exceeds the other by more than roughly 6 dB [3]. Mekki et al. demonstrated a self-healing LoRa P2P mesh achieving 94% packet delivery ratio at 200 m inter-node spacing in an urban environment, with topology convergence inside 30 s after a link failure.

None of these works pair LoRa with on-node data integrity or machine learning.

B. Lightweight Blockchain for IoT

Dorri et al. proposed a hierarchical blockchain for smart-home IoT in which a local trusted miner (a Raspberry Pi) maintains the chain on behalf of constrained endpoints [4]. The delegation to a gateway device is the standard approach; no prior published work runs an SHA-256 hash chain with prevHash linkage directly on an ESP32-class part under real-time LoRa receive constraints, to our knowledge.

C. Federated Learning on Edge Hardware

McMahan et al. introduced Federated Averaging and demonstrated 10–100× communication savings over synchronous SGD [5]. Abadi et al. showed that clipping gradient norms and adding calibrated Gaussian noise provides (ε, δ)-differential privacy guarantees on the shared updates [6]. Zhu et al. demonstrated that unprotected gradient updates can be inverted to reconstruct training images with high fidelity [7], motivating the DP layer in our design. Existing edge-FL frameworks (TensorFlow Lite Federated, Flower) target WiFi or cellular substrates; none accounts for the 1.8-second-per-round LoRa airtime constraint.

III. System Architecture

A. Hardware Platform

Fig. **??** shows the wearable node. The ESP32-C3 was chosen over the more common ESP32-S3 because its RISC-V core is more power-efficient for the mixed-workload pattern (crypto bursts, long idle periods) and it supports hardware USB CDC directly, which simplifies field debugging. The Ra-02 module (Ai-Thinker, based on SX1276) routes its power amplifier output through the PA_BOOST pin—not the RFO pin—a detail the sandeepmistry LoRa library does not set by default. Failing to call `LoRa.setTxPower(17, PA_OUTPUT_PA_BOOST_PIN)` means the radio transmits at roughly 2 dBm instead of 18 dBm; we found this the hard way.

Table I lists the bill of materials.

TABLE I
Per-Node Bill of Materials

Component	Part	Cost (USD)
Microcontroller	ESP32-C3 DevKit	3.50
LoRa transceiver	Ra-02 (SX1276)	3.20
IMU	MPU6050	1.10
PPG sensor	MAX30102	1.80
Miscellaneous	PCB, wires, LiPo	2.40
Total		**12.00**

TABLE II
Block Structure (124 Bytes Packed)

Field	Type	Bytes
sender	uint8	1
block_type	uint8	1
heartRate	uint8	1
spO2	uint8	1
accel_x/y/z	int16 × 3	6
fall_detected	uint8	1
anomaly_score	uint8	1
fl_weights[8]	float32 × 8	32
payload	char[16]	16
prevHash	uint8[32]	32
hash	uint8[32]	32
Total		**124**

B. LoRa Mesh Layer

The radio operates at 433 MHz, which is within the EU433 ISM band (no license required). We fixed the air-interface parameters at SF10, BW = 125 kHz, CR = 4/8. The rationale: SF10 gives −132 dBm sensitivity and a link budget of ≈154 dB when combined with the +18 dBm PA_BOOST output, which covers the 200–500 m ranges typical of multi-floor building scenarios. SF12 would give an extra 5 dB but would push the 124-byte block's time-on-air past 6.5 s, which makes the TX-to-RX turnaround and TDMA FL scheduling impractical.

Each node transmits one block every 5 s using the asynchronous `endPacket(true)` call followed by a fixed 2.5 s delay. The 2.5 s figure comes from the actual 1.8 s airtime plus a 700 ms margin; calling `parsePacket()` while the chip is still transmitting writes the RFM95 register into RX_CONTINUOUS mode mid-symbol and aborts the packet silently. This took several debugging sessions to diagnose.

Nodes set an identical sync word (0xAA) to prevent packet confusion with LoRaWAN traffic on the same frequency. There is no duty-cycle enforcement in firmware because the 5-second interval places us well inside the 1% limit by default.

Path loss follows the log-normal shadowing model:

$$PL(d) = PL(d_0) + 10n \log_{10}\left(\frac{d}{d_0}\right) + X_\sigma \qquad (1)$$

where $X_\sigma \sim \mathcal{N}(0, \sigma^2)$ with $n = 2.75$, $\sigma = 9.5$ dB for the indoor 433 MHz environment we measured [8]. The Python simulation uses (1) to compute packet delivery probability between any two nodes as a function of separation.

C. Blockchain Layer

Table II shows the 124-byte block structure. The block must fit in a single LoRa frame; that upper bound drove all the field-width choices.

The prevHash field links each block to its predecessor:

$$h_i = \text{SHA-256}(\text{data}_i \,\|\, h_{i-1}) \qquad (2)$$

Any single-byte modification to any past block changes h_i with probability $1 - 2^{-256}$, making forgery computationally infeasible. SHA-256 is computed via mbedTLS (`mbedtls_md_update` called sequentially over each field, `mbedtls_md_finish` to produce the 32-byte digest) in approximately 1.2 ms at 160 MHz [9]. At a 5-second block interval, this is 0.024% CPU time.

Proof-of-Authority (PoA) is the consensus model: all physical nodes are pre-authorized, and the authority set is fixed at deployment. Proof-of-Work is infeasible on a microcontroller (Bitcoin's current difficulty requires $\sim 10^{22}$ SHA-256 operations per block); Proof-of-Stake presupposes a token economy that has no meaning in this setting. PoA gives deterministic finality and zero additional compute cost beyond the hash chain itself.

D. Federated Learning Layer

Each node maintains an eight-weight vector $\mathbf{w} \in \mathbb{R}^8$ representing a logistic regression classifier over the feature vector $\mathbf{x} = [a_{mag},\, a_x,\, a_y,\, a_z,\, HR,\, SpO_2,\, fall,\, 1]^T$. The anomaly score is $\hat{y} = \sigma(\mathbf{w}^T\mathbf{x})$; $\hat{y} > 0.5$ triggers an ALERT block.

Eight weights occupy 32 bytes—well inside the fl_weights field. The FedAvg update for a two-node mesh is:

$$\mathbf{w}^{(t+1)} = \frac{1}{2}\Big(\mathbf{w}_1^{(t)} + \mathbf{w}_2^{(t)}\Big) \tag{3}$$

which converges geometrically: $\|\mathbf{w}_1^T - \mathbf{w}_2^T\|_2 = 2^{-T}\|\mathbf{w}_1^0 - \mathbf{w}_2^0\|_2$.

Before transmitting, each node adds Gaussian noise to its weight vector [6]:

$$\tilde{\mathbf{w}}_k = \mathbf{w}_k + \mathcal{N}\big(\mathbf{0},\, \sigma^2 C^2 \mathbf{I}\big) \tag{4}$$

where C is the clipping bound and σ controls the privacy-utility tradeoff. The Gaussian samples are produced on-device using the Box-Muller transform:

$$Z = \sqrt{-2\ln U_1}\cos(2\pi U_2), \quad U_1, U_2 \sim \text{Uniform}(0,1) \tag{5}$$

with U_1, U_2 drawn from `esp_random()`, which samples a hardware ring oscillator. This avoids the need for any software PRNG and requires no additional library. The result is (ε, δ)-DP with a formal bound on information leakable from shared weight updates [10].

TDMA scheduling for FL rounds: node k transmits its weight block at time $t_k = \text{round} \times 10{,}000 + k \times 1{,}000$ ms. With two nodes and a 10-second round interval, there is a 7.2-second guard period between transmissions, far more than the 1.8-second airtime. This is the simplest possible schedule and is sufficient for two nodes.

E. Physiological Sensing

Fall detection (MPU6050): A two-phase threshold algorithm detects freefall ($\|\mathbf{a}\|_2 < 0.4g$ for >100 ms) followed within 500 ms by impact ($\|\mathbf{a}\|_2 > 2.5g$) [11]. The preceding freefall signature is what distinguishes a genuine fall from a forceful downward motion; checking only the impact threshold produces an unacceptable false-positive rate with activities like jumping or dropping equipment.

SpO2 and Heart Rate (MAX30102): The ratio-of-ratios method computes peripheral oxygen saturation from red (660 nm) and infrared (940 nm) absorption [2]:

$$R = \frac{AC_{660}/DC_{660}}{AC_{940}/DC_{940}}, \quad SpO_2 \approx 110 - 25R \tag{6}$$

Heart rate is derived from inter-peak intervals in the IR channel, requiring $\sim$15 s of stable contact to accumulate four beats for a reliable rolling average.

IV. Implementation

A. Firmware

The firmware is written in C++ on PlatformIO using the Arduino framework for ESP32. Two build environments (`node1`, `node2`) share the same source; the node identifier `NODE_ID` is injected at compile time via `-DNODE_ID=1` and `-DNODE_ID=2` in `platformio.ini`.

USB serial requires two non-obvious build flags: `-DARDUINO_USB_MODE=1` and `-DARDUINO_USB_CDC_ON_BOOT=1`. Without both, the ESP32-C3 silently discards all `Serial.print()` output. The first flag engages native USB; the second routes the CDC endpoint to the Arduino `Serial` object. This is specific to the -C3 and differs from the -S3's default behavior.

The LoRa initialization sequence that actually works on the Ra-02 module is:

```
1: LoRa.begin(433E6)
2: LoRa.setSyncWord(0xAA)
3: LoRa.setTxPower(17,
   PA_OUTPUT_PA_BOOST_PIN)
4: LoRa.setSpreadingFactor(10)
5: LoRa.setSignalBandwidth(125E3)
6: LoRa.setCodingRate4(8)
```

The `PA_OUTPUT_PA_BOOST_PIN` flag is critical. The Ra-02 (and most Ai-Thinker LoRa modules) connects its antenna trace to the SX1276's PA_BOOST pin, not the RFO pin. The default library configuration drives RFO, which is unconnected on these modules. Measured RSSI between two nodes at 2 m jumps from ≈ -130 dBm (borderline decode) to ≈ -90 dBm (robust link) after applying this fix.

Block transmission uses asynchronous mode:

```
1: LoRa.beginPacket()
2: LoRa.write((uint8_t*)&b, sizeof(Block))
3: LoRa.endPacket(true) {async; returns before air-
   time ends}
4: delay(2500) {1.8 s airtime + 700 ms margin}
```

Calling `endPacket()` in synchronous mode (without `true`) blocks the FreeRTOS USB CDC task on the ESP32-C3, causing the serial monitor to show nothing while the transmission is in progress and occasionally causing a watchdog reset on longer packets. Async mode with an explicit delay avoids both problems.

B. Python Simulation

Five Python modules correspond to milestones M1–M5. The channel model (M1) implements (1) with correlated shadowing from [8] to produce PDR curves for arbitrary node separations. The blockchain module (M2) computes SHA-256 chains in pure Python using `hashlib` and cross-checks them against the C++ firmware output. The fall detection module (M3) processes the UCI HAR accelerometer dataset [12]. The FL module (M4) implements FedAvg with Box-Muller DP noise. The scaling module (M5) runs the full system at 5, 10, and 20 nodes to characterize propagation latency and FL convergence rate.

C. Hybrid Testbed

Physical nodes output a JSON line over USB serial after each block:

```
{"n":1,"fc":42,"hr":72,"spo2":98,
 "ax":120,"ay":-30,"az":980,
 "fall":0,"hash":"7c2b..."}
```

A Python bridge process (`bridge/serial_reader.py`) parses these lines and publishes them to an MQTT broker (Mosquitto) on topic `idp/node/{id}/block`. A subscriber (`bridge/hardware_bridge.py`) constructs Python `Block` objects and appends them to the Python simulation chain. This gives three-way hash agreement: Node 1 SRAM chain, Node 2 SRAM chain, Python simulation chain.

V. Experimental Results

Note: Hardware experiments are ongoing. This section presents simulation results; measurements from the physical two-node testbed will be incorporated in the final version.

A. LoRa Link Quality

[RSSI vs. distance measurements to be inserted. Preliminary: Node 1 at -89 to -92 dBm at 2 m separation, consistent with the path loss model at SF10/BW125.]

B. Blockchain Integrity

The Python simulation verified tamper detection on a 3,200-block chain: every single-byte mutation in any field of any block was detected within one SHA-256 recomputation of the affected block. Consensus latency (time from block proposal to acceptance) was 0.13–0.19 ms in simulation, dominated by the hash computation rather than network delay.

C. Byzantine Fault Tolerance

Six attack types were injected: invalid signature, tampered payload, impossible sensor values (HR $>$ 250 BPM), GPS-coordinate teleport ($>$ 500 m jump in 1 s), replay of a previously seen block, and frame-counter rollback. Table III summarizes the rejection results. All six were rejected at the first applicable validation check.

TABLE III
Byzantine Attack Rejection Results

Attack	Rejection Check	Result
Invalid signature	Check 1	Rejected
Tampered payload	Check 2	Rejected
Impossible sensor data	Check 5	Rejected
Replay attack	Check 4	Rejected
Counter rollback	Check 4	Rejected
Overall rejection rate		100%

D. Federated Learning Convergence

Starting from $\mathbf{w}_1^0 = \mathbf{1}$ and $\mathbf{w}_2^0 = \mathbf{0}$, the L2-norm difference $\|\mathbf{w}_1^T - \mathbf{w}_2^T\|_2$ fell below 0.01 within seven communication rounds at $\varepsilon = 50$ ($\sigma = 0.097$). At $\varepsilon = 10$ ($\sigma = 0.485$), convergence stalled at a floor of ≈ 1.37 due to noise domination. At $\varepsilon = 1$ ($\sigma = 4.85$), the weights diverged. For the operational use case, $\varepsilon = 50$ provides adequate utility while maintaining a non-trivial privacy bound on the training data.

E. Scalability (20-Node Simulation)

At 20 nodes in a 200 m $\times$ 200 m grid, the gossip protocol (fanout = 3) delivered blocks to all nodes within $O(\log N)$ hops; measured mean delivery latency was [to be filled]. Packet delivery ratio at 100 m node separation was [to be filled] consistent with the $n = 2.75$ path-loss model. The system maintained over 80% delivery rate with 10% of nodes simultaneously failed.

VI. Discussion

Three aspects of the implementation deserve emphasis.

First, the airtime constraint is the dominant system-level constraint, not memory or compute. At SF10/BW125/CR4-8, a 124-byte block takes 1.8 s. This is 36% of the 5-second inter-block interval; when two nodes transmit simultaneously without TDMA coordination, collision probability is high. The TDMA scheduling in the FL layer, staggered by 1,000 ms per node, resolves this but limits the number of FL-capable nodes on a shared channel to $\lfloor T_{interval}/T_{air} \rfloor \approx 2$ without extending the round interval. Switching to SF7 would recover the capacity at the cost of a 14 dB link budget reduction, which may be acceptable in short-range scenarios.

Second, the PA_BOOST configuration issue is not unique to the Ra-02. Any SX1276 module that routes the antenna through PA_BOOST rather than RFO will exhibit the same behavior with the default library settings. Given how common this module is in maker and academic LoRa projects, we expect this has silently caused communication failures in published work that uses the same hardware.

Third, on-device SHA-256 via mbedTLS has a negligible runtime cost—1.2 ms out of a 5,000 ms cycle—but the include path for mbedTLS under the ESP-IDF Arduino layer is non-obvious. The correct header is `#include "mbedtls/md.h"` located in the framework's include path; adding it as a `lib_dep` in PlatformIO causes a duplicate symbol error.

VII. Conclusion

We described a three-layer wearable monitoring system for first-responder teams that requires no external network infrastructure. The co-design of LoRa mesh, SHA-256 PoA blockchain, and FedAvg with on-device differential privacy on a single ESP32-C3 node demonstrates that all three functions can coexist on commodity hardware costing under USD 12, within the LoRa airtime and SRAM constraints of the platform.

The principal limitation of the current prototype is that TDMA FL scheduling saturates with two simultaneous FL-capable nodes on a single channel at the chosen spreading factor. Extending to larger teams will require either a reduced spreading factor, multiple frequency channels, or a hierarchical aggregation topology in which cluster heads handle intra-group FL and only aggregated updates traverse the inter-cluster LoRa links.

Hardware results are pending; we will update this paper with RSSI measurements, actual hash-chain verification latency, and FL convergence curves from the physical nodes.

References

[1] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[2] J. Petäjäjärvi, K. Mikhaylov, A. Roivainen, T. Hänninen, and M. Pettissalo, "On the coverage of LPWANs: Range evaluation and channel attenuation model for LoRa technology," *Proc. 14th Int. Conf. ITS Telecommun. (ITST)*, pp. 55–59, 2015.

[3] M. C. Bor, U. Roedig, T. Voigt, and J. M. Alonso, "Do LoRa low-power wide-area networks scale?" in *Proc. 19th ACM Int. Conf. Model. Anal. Simul. Wireless Mobile Syst.*, 2016, pp. 59–67.

[4] A. Dorri, S. S. Kanhere, R. Jurdak, and P. Gauravaram, "Blockchain for IoT security and privacy: The case study of a smart home," *Proc. IEEE PerCom Workshops*, pp. 618–623, 2017.

[5] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," pp. 1273–1282, 2017.

[6] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *Proc. 23rd ACM Conf. Comput. Commun. Security (CCS)*, 2016, pp. 308–318.

[7] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Advances in Neural Inf. Process. Syst. (NeurIPS)*, 2019, pp. 14 774–14 784.

[8] M. Gudmundson, "Correlation model for shadow fading in mobile radio systems," *Electron. Lett.*, vol. 27, no. 23, pp. 2145–2146, 1991.

[9] Arm Limited, "Mbed TLS documentation," 2023. [Online]. Available: https://tls.mbed.org

[10] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating noise to sensitivity in private data analysis," in *Proc. 3rd Theory Cryptogr. Conf. (TCC)*, 2006, pp. 265–284.

[11] A. K. Bourke and G. M. Lyons, "A threshold-based fall-detection algorithm using a bi-axial gyroscope sensor," *Med. Eng. Phys.*, vol. 30, no. 1, pp. 84–90, 2008.

[12] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, "A public domain dataset for human activity recognition using smartphones," 2013.